



UNIVERSIDAD AUTÓNOMA DE AGUASCALIENTES
CENTRO DE CIENCIAS BÁSICAS
DEPARTAMENTO DE SISTEMAS ELECTRÓNICOS
INGENIERÍA EN SISTEMAS COMPUTACIONALES • COMPILADORES II

FASE 3 DEL PROYECTO DE COMPILADORES

ANÁLISIS SEMÁNTICO

Materia	Compiladores II
Semestre	9.º ISC
Docente	Dra. Blanca Guadalupe Estrada Rentería
Fecha de entrega y revisión	15 y 16 de octubre de 2026, durante el horario habitual de clase y de acuerdo con la calendarización entregada vía Aula Virtual.

1. Introducción

En esta fase se incorporará el análisis semántico al compilador que el equipo ha venido desarrollando. Se trabajará directamente con el Árbol Sintáctico Abstracto (AST) obtenido en la fase sintáctica, agregando la información necesaria para verificar que las construcciones reconocidas por la gramática también sean semánticamente válidas.

El análisis semántico utiliza la estructura sintáctica del programa para calcular información adicional relacionada principalmente con los atributos, los tipos de datos, los identificadores y la tabla de símbolos.

Por lo tanto, el trabajo de esta fase se concentrará fundamentalmente en:

AST → reglas semánticas → atributos → árbol anotado → tabla de símbolos → verificación de tipos → detección de errores semánticos

2. Objetivo de la fase

Diseñar, implementar e integrar el analizador semántico al compilador, utilizando el AST generado por el analizador sintáctico para aplicar las reglas semánticas del lenguaje, asignar y propagar atributos, construir y consultar la tabla de símbolos, verificar la compatibilidad de tipos y detectar los errores semánticos presentes en el programa fuente.

Al finalizar la fase, el compilador deberá poder determinar si las construcciones que ya fueron reconocidas sintácticamente son también válidas desde el punto de vista semántico.

3. Requisitos previos

Antes de iniciar esta fase deberá encontrarse funcionando correctamente el compilador desarrollado hasta el análisis sintáctico.

En particular, se requiere que:

- el archivo de código fuente pueda ser procesado por el compilador;
- el analizador léxico genere correctamente los tokens;
- el analizador sintáctico procese dichos tokens;
- se construya correctamente el Árbol Sintáctico Abstracto (AST);
- se conserve la información necesaria de los identificadores y tokens, particularmente lexema, línea y columna cuando esté disponible;
- continúe funcionando el manejo de errores correspondiente a las fases anteriores.

La fase sintáctica ya establece que el analizador debe procesar los tokens y construir el AST del programa.

No deberán rehacerse los componentes anteriores; únicamente deberán realizarse las adecuaciones necesarias para incorporar el análisis semántico.

4. Desarrollo de la fase

4.1 Análisis de las construcciones de la gramática

Partan de la gramática que ya utiliza su compilador.

La gramática desarrollada en la fase sintáctica contempla declaraciones de variables, tipos int, float y bool, asignaciones, expresiones aritméticas, expresiones relacionales y lógicas, estructuras de selección, iteración y repetición, así como sentencias de entrada y salida.

Para cada construcción deberán identificar:

- qué información semántica necesita;
- qué atributos intervienen;
- de dónde se obtiene cada atributo;

- hacia dónde debe propagarse;
- qué información debe consultarse en la tabla de símbolos;
- qué condiciones deben cumplirse para considerar válida la construcción;
- qué situaciones deberán producir un error semántico.

No deberán modificar la gramática únicamente para facilitar la implementación de esta fase.

4.2 Reglas semánticas y atributos

Las producciones de la gramática que requieran información adicional deberán complementarse con sus correspondientes reglas semánticas.

En la semántica dirigida por la sintaxis, los atributos se asocian con los símbolos gramaticales y las reglas semánticas especifican la forma en que dichos atributos se calculan o se relacionan.

El equipo deberá identificar y utilizar, según corresponda:

Atributos sintetizados

La información se obtiene a partir de los nodos descendientes y se propaga hacia los nodos superiores.

En esta fase son especialmente importantes para determinar, por ejemplo, el tipo resultante de una expresión.

Atributos heredados

La información procede del contexto y se transmite hacia otros elementos relacionados con la construcción.

Un ejemplo directamente relacionado con el proyecto se encuentra en las declaraciones de variables: el tipo indicado en la declaración debe propagarse hacia los identificadores declarados. Los materiales de la asignatura muestran este comportamiento mediante el atributo dtype.

El equipo deberá documentar claramente las reglas semánticas que realmente fueron incorporadas a su compilador.

5. Árbol sintáctico con anotaciones semánticas

El AST desarrollado en la fase sintáctica será la estructura base de esta fase.

Se deberá construir una copia del árbol sintáctico generado por el compilador.

A partir del AST anterior deberán incorporarse las anotaciones semánticas necesarias para representar la información calculada durante el análisis.

La versión anterior de esta fase establece expresamente que debe partirse del AST y agregarse información semántica, mostrando además cómo se propagan los atributos entre los nodos.

El árbol anotado deberá permitir identificar, cuando corresponda:

- identificadores;
- tipos declarados;
- tipos asociados a identificadores;
- tipos obtenidos para expresiones;
- resultados de la propagación de atributos;
- información necesaria para realizar la verificación semántica.

Atributo de tipo

El tipo será uno de los atributos fundamentales de esta fase. Deberá calcularse y propagarse de acuerdo con las reglas correspondientes a cada construcción.

Atributo de valor

Cuando el valor pueda determinarse durante el análisis deberá asociarse con el nodo correspondiente.

Debe considerarse que los valores de muchas expresiones se determinan durante la ejecución; sin embargo, existen expresiones constantes cuyo valor puede calcularse durante la compilación.

Por ello, no se deberá forzar la obtención de un valor cuando éste no pueda determinarse en esta etapa.

Evidencia del árbol anotado

Durante la revisión deberá ser posible comparar:

AST original → AST con anotaciones semánticas

de manera que pueda observarse claramente qué información fue incorporada durante esta fase y cómo se propagaron los atributos utilizados.

6. Tabla de símbolos

La tabla de símbolos deberá formar parte del funcionamiento real del analizador semántico.

No deberá elaborarse únicamente como una tabla de presentación o como evidencia independiente.

La tabla de símbolos almacena información asociada con los identificadores del programa y permite posteriormente recuperar dicha información mediante operaciones de inserción y búsqueda. En los materiales de análisis semántico se describe como una estructura tipo diccionario que puede utilizar operaciones de inserción, búsqueda y eliminación.

6.1 Construcción de la tabla

Al procesar una declaración, el analizador deberá incorporar a la tabla los identificadores correspondientes y asociarles la información semántica obtenida.

La fase deberá registrar, de acuerdo con la estructura utilizada en el proyecto:

- Nombre o lexema del identificador.
- Tipo de dato.
- Números de línea donde se localizaron los identificadores.
- Dirección o desplazamiento de memoria, conforme a la estructura definida para la tabla del proyecto.

Estos elementos se establecieron también como parte de los requerimientos de la fase semántica utilizada previamente.

6.2 Operaciones sobre la tabla

La implementación deberá permitir utilizar la tabla durante el análisis, principalmente para:

- insertar identificadores declarados;
- buscar identificadores cuando sean utilizados;
- obtener el tipo asociado a un identificador;
- verificar si un identificador ya ha sido declarado;
- detectar identificadores no declarados;
- detectar declaraciones duplicadas;
- manejar correctamente el ámbito definido por el lenguaje.

La información de una declaración debe introducir el identificador y su tipo en la tabla; posteriormente, las expresiones y sentencias pueden recuperar ese tipo mediante una búsqueda.

6.3 Relación entre el AST y la tabla de símbolos

El árbol anotado y la tabla de símbolos no son productos independientes.

Durante el recorrido del AST:

Declaración → inserción en tabla

Uso de identificador → búsqueda en tabla

Expresión → obtención y propagación de tipos

Asignación → comparación del tipo del identificador con el tipo de la expresión

Esta relación deberá poder observarse durante la demostración del funcionamiento.

7. Verificación y conversión de tipos

La verificación de tipos constituye una de las actividades fundamentales del análisis semántico.

El compilador deberá determinar el tipo de los identificadores y expresiones y comprobar que sean compatibles con la construcción en la cual están siendo utilizados.

La comprobación de tipos sigue la estructura operador/operando del árbol sintáctico y permite verificar que el tipo calculado coincida con el esperado por el contexto.

El analizador deberá verificar, cuando corresponda:

- compatibilidad entre los operandos de una expresión;
- tipo resultante de las expresiones;
- compatibilidad entre el identificador y la expresión de una asignación;
- compatibilidad de los operadores con los tipos de sus operandos;
- tipo de las expresiones utilizadas en las estructuras de control;
- conversiones de tipos permitidas;
- conversiones de tipos no válidas.

Conversiones de tipos

Cuando los operandos sean de tipos diferentes, deberá aplicarse la regla correspondiente definida para el lenguaje.

Los materiales muestran, por ejemplo, la posibilidad de convertir un entero a punto flotante cuando se combina con un operando float.

El equipo deberá poder explicar:

- qué conversiones acepta su analizador;
- en qué situación se realizan;
- cómo queda representada dicha información en el árbol anotado;
- cuándo una combinación de tipos debe generar un error semántico.

8. Manejo de errores semánticos

Los errores detectados en esta fase deberán corresponder específicamente al análisis semántico.

Como mínimo, el analizador deberá identificar, cuando las construcciones correspondientes aparezcan en el código fuente:

1. Uso de variables o identificadores no declarados.
2. Duplicidad de identificadores o declaraciones dentro del ámbito correspondiente.
3. Incompatibilidad de tipos.
4. Conversión de tipos no válida.
5. Uso indebido de operadores de acuerdo con los tipos de sus operandos.
6. Inconsistencias de tipos en asignaciones y expresiones.
7. Tipo incorrecto en una condición cuando la construcción requiera una expresión booleana.

Estos errores forman parte de los requerimientos establecidos para el analizador semántico del proyecto.

Los errores deberán mostrarse en el espacio destinado para ello dentro del IDE.

Desde el diseño original del ambiente de desarrollo se estableció una lista independiente para errores léxicos, sintácticos y semánticos, incluyendo número de línea y una descripción clara.

Cuando la información de columna se encuentre disponible desde las fases anteriores, deberá conservarse.

Siempre que el error detectado no impida continuar de forma segura el análisis, deberá procurarse continuar el recorrido para identificar otros errores semánticos presentes en el programa.

9. Requerimientos de funcionamiento de la fase

Al finalizar el desarrollo, el analizador semántico deberá ser capaz de realizar de manera integrada el siguiente proceso:

1. Recibir el AST generado por el analizador sintáctico.
2. Recorrer las construcciones que requieran procesamiento semántico.
3. Aplicar las reglas semánticas correspondientes.
4. Asignar y propagar atributos sintetizados y heredados cuando sean necesarios.
5. Incorporar las anotaciones correspondientes al AST.
6. Construir y actualizar la tabla de símbolos a partir de las declaraciones encontradas.
7. Consultar la tabla de símbolos cuando aparezcan identificadores en expresiones o sentencias.
8. Determinar y propagar los tipos de las expresiones.
9. Verificar la compatibilidad de tipos y las conversiones permitidas.
10. Detectar y reportar los errores semánticos.
11. Mostrar en el IDE los resultados correspondientes al análisis semántico, el árbol anotado, la tabla de símbolos y los errores detectados.

El IDE diseñado para el proyecto ya contempla específicamente la visualización del resultado semántico, la tabla de símbolos y la lista de errores.

10. Pruebas de funcionamiento

Para la validación de esta fase se utilizará el mismo archivo de código fuente con el que se ha trabajado desde el semestre anterior.

No deberán diseñar un lenguaje diferente ni sustituir el archivo principal de pruebas por otro programa elaborado únicamente para esta fase.

En caso de que algún equipo ya no cuente con dicho archivo, estará disponible en el Aula Virtual.

A partir de este archivo deberá demostrarse el funcionamiento completo del análisis semántico.

Durante las pruebas deberá poder observarse:

- el AST obtenido previamente;
- el recorrido o procesamiento semántico correspondiente;
- la incorporación de las anotaciones semánticas;
- la propagación de atributos;
- la construcción de la tabla de símbolos;
- la búsqueda de identificadores dentro de la tabla;
- la determinación de tipos;
- la verificación de compatibilidad;
- las conversiones aplicadas cuando procedan;
- la detección de los errores semánticos presentes;
- el reporte correspondiente dentro del IDE.

El propósito de esta prueba es verificar que el compilador realmente realice el análisis sobre el código fuente que se ha venido utilizando a lo largo del proyecto, manteniendo continuidad entre las distintas fases.

11. Evidencias y entregables

La entrega deberá corresponder a la versión funcional del compilador que ya incorpora el análisis semántico.

Cada equipo deberá entregar:

1. Código fuente completo y actualizado del compilador, incluyendo la implementación del análisis semántico.
2. Árbol Sintáctico Abstracto utilizado como base del análisis.

3. Árbol con anotaciones semánticas, en el que puedan observarse los atributos incorporados y su propagación.
4. Tabla de símbolos generada durante el análisis, mostrando claramente la información correspondiente a los identificadores.
5. Reporte o archivo de errores semánticos detectados.
6. Evidencias de las pruebas de funcionamiento, incluyendo capturas suficientes para comprobar:
 - árbol anotado;
 - tabla de símbolos;
 - comprobación de tipos;
 - errores semánticos;
 - resultado mostrado en el IDE.
7. Documento PDF de la fase, que deberá contener de manera clara y ordenada:
 - descripción de la implementación realizada;
 - reglas semánticas utilizadas;
 - atributos sintetizados y heredados implementados;
 - explicación de la forma en que se anotó el AST;
 - diseño y funcionamiento de la tabla de símbolos;
 - reglas utilizadas para la comprobación de tipos;
 - conversiones de tipos consideradas;
 - errores semánticos implementados;
 - explicación de las pruebas realizadas;
 - capturas de ejecución.
8. Archivo de texto con los nombres completos y matrículas de todos los integrantes del equipo.

Los productos principales mantienen la estructura establecida previamente para esta fase: código fuente, árbol sintáctico y árbol anotado, tabla de símbolos, errores semánticos y documentación técnica.

La entrega en el Aula Virtual será una sola por equipo, integrando todos los elementos solicitados.

12. Fecha de entrega y revisión

La fase de Análisis Semántico se entregará y revisará los días:

15 y 16 de octubre de 2026

La presentación de cada equipo se realizará durante el horario habitual de clase, de acuerdo con la calendarización entregada vía Aula Virtual.

Nota importante sobre la entrega

- La fase deberá entregarse completa y funcional.
- Por cada día de retraso se aplicará una penalización del 10 % sobre la calificación de la fase.
- La penalización por retraso no exime al equipo de realizar la entrega completa.
- Aun cuando la acumulación de penalizaciones afecte totalmente la calificación obtenida, la entrega de la fase continuará siendo obligatoria.

Este criterio mantiene la condición establecida desde las primeras fases del proyecto: la fase se considera completa cuando cumple los requerimientos solicitados y cada día de retraso implica una penalización del 10 %.